

AI Landscape Tracker — System Architecture

How the pipeline scales to 500K+ records without manual work, handles provider failures gracefully, avoids duplicate news/job entries, and what data stores back the whole system.

1. Scaling to 500,000+ Records Without Manual Work

The current pipeline (Section overview: **YC/Crunchbase** → **Startups**, **ProductHunt** → **Products**, **arXiv API** → **Papers**, **5 news sites + 5 job boards** → **fresh content**) is built to scale horizontally rather than by making any single scraper 'smarter'. Three mechanisms make 500K+ rows achievable unattended:

- **Source fan-out, not deeper single-source scraping.** Each entity type (startup, product, paper, news, job) is scraped by an independent async module. Adding a new source is adding one new file that emits the same Pydantic schema — the orchestrator (`main.py`) simply gathers one more coroutine with `asyncio.gather()`. Going from 5 job boards to 50 is a config change, not a rewrite.
- **Pagination-first, not scroll-first, where an API exists.** arXiv, GitHub, and (when available) ProductHunt's GraphQL API and Crunchbase's REST API are paged programmatically (offset/cursor loops) rather than relying on browser scrolling, which is far more reliable at high volume and doesn't need a headless browser instance per request.
- **Horizontal worker scaling via a job queue.** At 500K+ rows, the single-process `asyncio.gather()` model is replaced by pushing '(source, page/cursor)' work units onto a lightweight queue (Redis + RQ, or Celery). N worker containers pull units independently, so total throughput scales linearly with worker count. Each worker still uses the same `HttpClient` retry/backoff logic — scaling out doesn't change per-request correctness.
- **Scheduled incremental runs, not one giant batch.** A cron/GitHub Actions job runs scrapers every N hours pulling only new/changed records (e.g. arXiv 'submittedDate desc' page-1 only, GitHub stars refreshed for repos not touched in 7 days). This keeps steady-state runs cheap even once the historical dataset is huge.

2. Handling 413 and 429 Errors

413 (Payload / Context Too Large) — prevented proactively, not just caught reactively:

- Every scraped page/article is capped at a hard byte ceiling while streaming (`MAX_RESPONSE_BYTES` in `utils/http_client.py`) — oversized responses abort before they ever reach parsing.
- Before any text reaches an LLM, `llm/fallback_chain.chunk_text()` splits it on paragraph boundaries into ~12,000-character chunks (a conservative ~3K-token budget), hard-splitting any single paragraph that alone exceeds the limit. Each chunk is extracted independently and results are merged by the caller.

429 (Rate Limited) — handled at two layers:

- **HTTP layer:** `HttpClient.get_text()` uses tenacity for exponential backoff with jitter (2s → 60s cap, 5 attempts), and honors a server's `Retry-After` header when present.
- **LLM layer:** each provider gets up to 2 backoff-retried attempts (`_call_with_backoff`); if still failing, the fallback chain moves to the next provider automatically — Gemini Flash → Groq (Llama 3) → DeepSeek — so a single provider's rate limit never stalls the whole pipeline.

3. Deduplication Strategy for News & Jobs

News and job postings are re-scraped every run, so the same article/posting will be seen multiple times across runs unless it's explicitly tracked. The strategy has three layers:

- **Deterministic content fingerprint as primary key.** Each row's dedup key is `sha256(normalized_url)` where `normalized_url` strips tracking query params (`utm_*`, `ref`) and trailing slashes. URL is the most stable unique identifier a news/job site gives us.
- **Secondary fuzzy key for syndicated content.** The same story is sometimes republished at a different URL (e.g. a wire story picked up by two sites). A secondary key — (`source_domain_family`, `RapidFuzz token_sort_ratio(title) >= 92`, published within 2h of each other) — catches these near-duplicates using the same fuzzy-matching machinery already built for entity resolution (`entity_resolution/resolver.py`), reused here against titles instead of company names.
- **'Seen before' table checked before insert, not after.** Before writing a row, the pipeline checks the fingerprint against a persistent `seen_urls` table (see DB choice below) with a 30-day retention window. Only genuinely new fingerprints are written to that run's output and, downstream, to the Google Sheet — so the sheet is always append-only-of-new, never re-populated with stale duplicates on each run.

4. Database Choice

Primary datastore: PostgreSQL.

- All five entity types are naturally relational (a Product belongs to a Startup, a Job belongs to a Company, a Paper links to a GitHub repo) — foreign keys enforce these relationships instead of relying on string-matching at read time.
- Native UNIQUE constraints on (`source`, `url_fingerprint`) give the dedup layer above a hard guarantee, not just an application-level convention — a duplicate insert simply fails instead of silently succeeding.
- JSONB columns hold semi-structured LLM extraction output (e.g. raw fields the schema hasn't formalized yet) without needing a schema migration for every new field a source happens to provide.
- Mature, free-tier-friendly managed hosting (Supabase / Neon / RDS free tier) keeps this realistic for a project of this scope, while still being the same engine used in production systems at much larger scale.

Secondary: a vector store (pgvector extension on the same Postgres instance).

- Startup/product descriptions and paper abstracts are embedded (e.g. `text-embedding-3-small` or a local model) and stored in `pgvector` columns. This directly improves entity resolution: a borderline fuzzy-match case (RapidFuzz score in the 75–90 ambiguous band) can be cross-checked with cosine similarity on description embeddings before falling back to an LLM tie-breaker call, cutting LLM cost for the ambiguous band significantly at 500K+ scale.
- It also directly enables product features beyond this assignment's scope — semantic search ('find startups similar to X') — without introducing a second database system to operate, since `pgvector` lives inside the same Postgres instance.

Why not a separate graph DB (e.g. Neo4j): the entity relationships here (Startup→Product, Company→Job, Paper→Repo) are shallow (1–2 hops), which standard SQL joins and a `startup_id` foreign key handle cleanly. A dedicated graph engine would add operational overhead without a corresponding query-complexity payoff at this stage; it's a reasonable future addition only if the roadmap grows toward deep multi-hop relationship queries (e.g. investor→startup→acquirer chains).

Pipeline Summary

Stage	Tooling	Output
Scrape	aiohttp / Playwright / arXiv API / GitHub API	Raw rows per source
Structure	Gemini → Groq → DeepSeek fallback chain	Validated Pydantic JSON
Resolve	RapidFuzz + LLM tie-breaker	Canonical names + audit log
Store	PostgreSQL + pgvector	Deduplicated, queryable dataset
Publish	gsread (Google Sheets API)	6-tab public-view spreadsheet